

Making the Qubit Tune Itself: Closed-Loop Calibration of a Drifting Three-Level Transmon

Q-Blueprint, QARC IIT (ISM) Dhanbad, Concetto'26

Summary

This report describes a simulator of a drifting three-level transmon, an autonomous agent that keeps its single-qubit gates calibrated using only shot counts, and a measurement of what different calibration policies do to a $p = 1$ QAOA on a 4-qubit ring. Everything here comes from simulation, with fixed seeds. We ran 22 drift traces (the official seed 2026 plus seeds 1 to 21) against 22 policy configurations, which makes 484 simulated days. Where we quote an error bar we say whether it is a standard deviation (SD), a standard error (SE) or a 95% confidence interval (CI) of the mean.

The main results, in short:

- With the specified truncated Gaussian pulse, even a perfectly calibrated π pulse has a gate error of 1.75×10^{-4} . Averaged over the π and $\pi/2$ gates the floor is about 0.94×10^{-4} .
- Calibration is cheap. One recalibration takes roughly 520 shots, about 0.26 s of device time. Recalibrating every 10 minutes uses only 0.044% of the day.
- Recalibrating once an hour (policy P1) gives a time-averaged gate error of 2.15×10^{-4} ($\text{CI} \pm 0.23 \times 10^{-4}$), against 8.84×10^{-4} for never recalibrating (P0).
- Our health-check policy (P2) gave no clear gain over simple periodic recalibration when its shots are counted as downtime. We report this as a negative result.
- The QAOA is forgiving. Even with no recalibration it loses only 0.23% of its ideal cost on average. The frequency jump hurts it most, while gain drift matters most for raw gate error.

1. The model

We use the three-level Hamiltonian from the problem statement, in the frame rotating at the drive frequency:

$$H(t) = \Delta(t) a^\dagger a + \frac{\alpha}{2} a^\dagger a^\dagger a a + \frac{g(t)}{2} [\Omega_x(t)(a + a^\dagger) + \Omega_y(t)i(a^\dagger - a)]$$

Here a lowers the level index, so $a^\dagger a = \text{diag}(0, 1, 2)$ and $a^\dagger a^\dagger a a = \text{diag}(0, 0, 2)$. The diagonal of H is therefore $(0, \Delta, 2\Delta + \alpha)$, and the $1 \rightarrow 2$ transition sits at $\Delta + \alpha$, about 300 MHz away from the drive. The drive couples $1 \leftrightarrow 2$ a factor $\sqrt{2}$ more strongly than $0 \leftrightarrow 1$, which is why leakage matters at all. Inside the qubit block, $a + a^\dagger$ acts as σ_x and $i(a^\dagger - a)$ as σ_y .

Units. We work in nanoseconds and rad/ns internally, with $\Delta = 2\pi \times$ (frequency in Hz). Hz appears only at the input and output. A unit test checks that 1 MHz acting for 250 ns gives a phase of $\pi/2$.

Pulse. The x-quadrature is a Gaussian of width $\sigma = t_g/4 = 5$ ns, truncated to the 20 ns window, with its area over that window set to the rotation angle. We did not subtract a baseline, so the envelope jumps to 13.5% of its peak at the edges. This is what the problem specifies, and it turns out to set a leakage floor (Section 2). The DRAG term is $\Omega y = -\beta d\Omega x/dt$. A first-order argument (integrating the $1 \rightarrow 2$ amplitude by parts) predicts that leakage cancels at $\beta = 1/\alpha$, which is negative here since $\alpha < 0$, with magnitude $1/|\alpha| = 0.53$ ns. As we show next, the real optimum is not at that value.

Propagator. Each gate is a product of 3×3 matrix exponentials, one per time step, computed from the eigendecomposition of the Hermitian H . Later steps multiply on the left. We use 200 steps of 0.1 ns for the plant and 100 steps in the evaluator. The error falls by a factor of 4.0 each time we double the step count, as expected for the midpoint rule.

Measurement. Every shot starts in $|0\rangle$. Readout reports 1 with probability 1% from $|0\rangle$ and 97% from $|1\rangle$, and $|2\rangle$ reads as 1. Counts are drawn from a binomial distribution. During idle periods the simulator applies the coherent phase, T_1 decay and a coherence decay of $e^{(-\tau/T_2)}$. We do not model decoherence during the 20 ns pulses, and the oracle gate error (Eq. 3 of the problem) is purely coherent.

Keeping the agent honest. The simulated device only exposes a `run(circuit, shots, controls)` call that returns integer counts, plus a clock and a shot counter. The agent can change three controls: a frame-frequency correction, an amplitude scale and the DRAG coefficient. These change the pulse, not the transmon: the effective detuning is Δ_{true} minus the correction, and the effective rotation is g times the amplitude scale. The oracle, the QAOA code and the drift generators are not imported by any agent code. A test parses the agent files and fails if that changes, if the agent touches a private attribute of the device, or if it stores a drift trace.

2. Static baseline

With $\Delta = 0$, $g = 1$ and an area of π , a pulse without DRAG takes $|0\rangle$ to $|1\rangle$ with probability 0.99628, leaks 1.75×10^{-4} into $|2\rangle$, and has a gate error of 2.54×10^{-3} (Figure 1). The error is much larger than the leakage, so most of it is coherent. Scanning β showed that leakage barely moves, because it is set by the pulse truncation, while the gate error has a sharp minimum at $\beta = -0.313$ ns (with amplitude scale 1.00133), where it drops to 1.75×10^{-4} . That is about 0.59 of the first-order value, not 1. Our reading is that most of the $\beta = 0$ error is an AC-Stark tilt of the rotation axis caused by the $|2\rangle$ level, and DRAG cancels that tilt well before it cancels leakage. The $\pi/2$ gate reaches 1.4×10^{-5} at the same settings. We also checked the oracle against the closed form $\varepsilon = (2/3)\sin^2(\delta\theta/2)$ for a pure over-rotation, and confirmed that leakage adds to the error.

Fig. 3 Static Gaussian pi pulse (Delta=0, g=1): populations during the gate, starting in $|0\rangle$. Exact propagation, 400 steps.

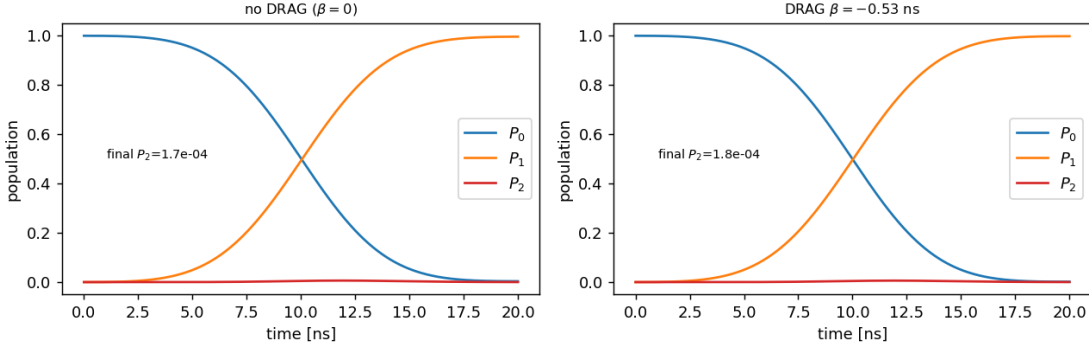


Figure 1. Populations during a static π pulse, without DRAG (left) and with $\beta = -0.53$ ns (right).

3. Calibration primitives

Amplitude. We start with a coarse Rabi sweep, then apply $RX(\pi/2)$ followed by n nominal π pulses, with n in $\{0, 1, 2, 4, 8, 16\}$. The signal is $\cos((n + \frac{1}{2})\pi(1 + \eta))$. The problem's Eq. 4 has n in place of $n + \frac{1}{2}$, because it leaves out the error of the initial $\pi/2$ pulse; we use the exact form. Repeating the pulse multiplies the sensitivity to η by about n , until higher-order errors or decoherence take over. Sixteen pulses last $0.3 \mu s$, well below T_2 , so decoherence is not the limit here. Over 60 random gains, the RMS error of g times the amplitude scale fell from 9.3×10^{-3} at 60 shots to 1.4×10^{-3} at 1920 shots, against 6×10^{-2} down to 9×10^{-3} for the Rabi sweep alone. A residual error of 1.4×10^{-3} costs only about 10^{-5} in gate error.

Frequency. For each delay we run two Ramsey circuits that differ by a known phase offset (0 and $\pi/2$), so that the phase comes from an arctangent and keeps its sign. Delays follow a ladder $T_k = T_0 \cdot 2^k$ with $T_0 = 300$ ns, and each rung is unwrapped using the estimate from the one before. The measured phase depends on the pulse-centre separation $\tau + t_g$, not just τ , because the qubit also accumulates phase during the pulses; we checked this against the simulator. A single delay T_0 is unambiguous only for $|\Delta| < 1/(2T_0) = 1.67$ MHz. We tested the limit directly: a true detuning of 1.9 MHz comes out aliased to $1.9 \text{ MHz} - 1/T_0$. Within range, the RMS error fell from 2.2 kHz at 160 shots to 0.45 kHz at 2560 shots over 80 random detunings, and about 95 to 100% of runs fell within two reported standard deviations.

DRAG. A binary readout cannot tell $|1\rangle$ from $|2\rangle$, so leakage is invisible directly. We use identity pairs, $[X(\pi) X(-\pi)]^n$ with $n = 8$, and read $P(1)$. The pair cancels any error that is odd in the sign of the drive, so what is left is the even-order error, meaning the Stark tilt and leakage, and leaked population shows up as extra readings of 1. Scanning β gives a fringe of the form $\sin^2(n c (\beta - \beta_0))$ with c close to 0.38 per ns, and we fit it by maximum likelihood. Our first fit left c free and occasionally landed on a wrong fringe, giving RMS errors of 0.6 to 1 ns. Bounding c and β_0 to their physical range fixed this: the RMS error against the noise-free fixed point (-0.3106 ns) fell from 0.011 ns at 275 shots to 0.0027 ns at 4400 shots, with 92 to 100% two-sigma coverage. That fixed point differs from the true gate-error minimum (-0.3134 ns) by an amount that costs 2×10^{-7} in gate error. One more thing we learned: the fixed point moves by

about 0.016 ns for every 100 kHz of leftover detuning, so DRAG has to be calibrated after frequency.

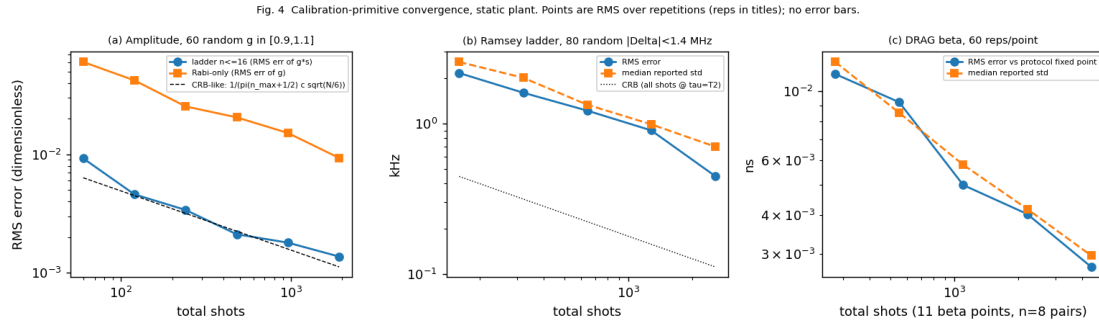


Figure 2. Convergence of the three primitives on a static plant (RMS over repeated runs).

4. Adaptive Bayesian frequency estimation

The agent keeps a posterior over Δ on a grid of ± 2 MHz. After each batch of shots it multiplies in the binomial likelihood, renormalises, and picks the next delay and phase offset by greedy expected information gain over 32 candidates. We compared this against a fixed log-spaced grid of 24 delays and a fixed uniform grid of 48 delays, using exactly the same number of total shots and the same likelihood.

We had to change how we summarise the posterior. Our first version reported the plain posterior standard deviation, and it was dominated by tiny leftover mass (around 4×10^{-4}) on far aliases, so it claimed 20 kHz when the actual error was under 1 kHz. We now report the mean and spread of the dominant mode together with the probability mass outside it, which we call the alias risk. Both strategies are scored with the same estimator.

With 100 random true detunings per budget:

Shots	Adaptive RMS (kHz)	Fixed log grid RMS (kHz)	Fixed uniform grid RMS (kHz)
100	25.0 (1% failures)	2.33	1640 (99% fail)
200	200 (1% failures)	1.43	1617
400	0.526	0.854	1620
800	0.243	0.593	1415
1600	0.165	0.415	1348
3200	0.122	0.326	1286 (84% fail)

To reach 0.5 kHz RMS the adaptive scheme needs about 420 shots and the fixed log grid about 1,110, a saving of 2.7 times (3.5 times at 0.4 kHz). The information gained per shot is only 6 to 14% higher for the adaptive scheme (0.094 against 0.089 bits per shot at 100 shots), so the saving mostly comes from spending shots on long, informative delays, which climb from about 0.45 μ s to 80 μ s as the posterior narrows, instead of on short ones. The uniform grid fails because it aliases. The adaptive scheme is not better everywhere: with 200 shots or fewer it fails

about 1% of the time, and the RMS in those rows is dominated by single runs that picked the wrong alias. For tiny budgets the fixed grid is the safer choice.

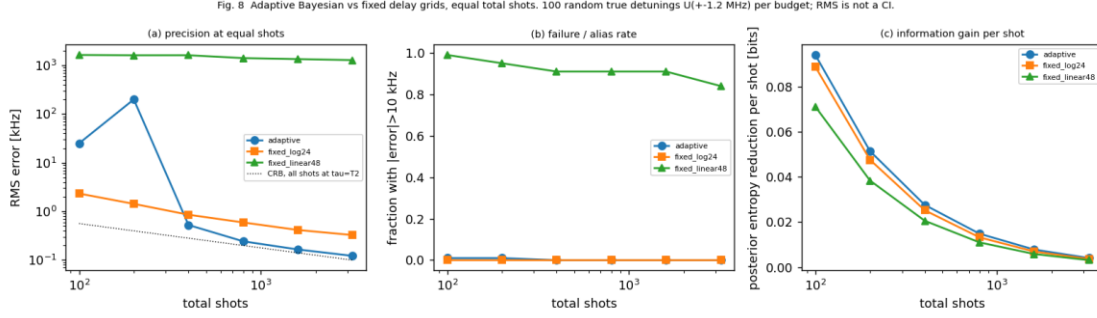


Figure 3. Adaptive and fixed designs at equal total shots.

5. The agent and the policies

The agent’s belief is its current controls, its uncertainties, the time of the last calibration and a log of its experiments. A cold calibration runs Rabi, then Bayesian Ramsey with a flat prior, then the amplitude ladder, then the DRAG scan, then a final amplitude touch-up; it takes about 2,600 shots (1.3 s; 2,580 on the official seed). A warm recalibration reuses the previous state and takes about 520 shots (0.26 s): Bayesian Ramsey with a prior based on how far an OU process can drift in the elapsed time (plus 15% broad weight in case of a jump), followed by a short amplitude ladder with $n = 1, 4, 16$.

The three policies are P0 (cold calibration only), P1 (warm recalibration every T minutes, with T from 10 to 720) and P2 (a health check every few minutes, with a warm recalibration only when a statistic crosses a threshold). Every shot advances the clock by 500 μs , and calibration and health-check shots, including the reference measurement, count as device downtime.

Our first health check used three sub-experiments of 150 shots every 5 minutes. It fired 148 recalibrations in one day and cost more than the calibrations it gated, because it was far more sensitive than the gates need. To pick a sensible sensitivity we measured how gate error actually depends on the errors (with the oracle). The error grows roughly as 1.3×10^{-3} times (residual detuning in MHz) squared, so 0.3 MHz adds only 1.2×10^{-4} . A rotation error of 1% adds 1.1×10^{-4} and 2% adds 4.2×10^{-4} . A DRAG error of 0.05 ns adds about 3×10^{-5} . We therefore rebuilt the check around those tolerances: a Ramsey circuit at 0.3 μs and an amplification circuit with $n = 8$, each run for 40 or 100 shots, compared with a reference taken straight after calibration using a two-sample chi-squared statistic with two degrees of freedom. The threshold comes from a chosen false-alarm probability. We tested that the false-alarm rate with no drift is below 2% over 300 checks, and that the check fires on a 0.8 MHz jump and a 4% gain step.

6. Policy comparison

We score each policy by (A) its time-averaged oracle gate error, the mean of the π and $\pi/2$ gate errors over the time the device is available, and (B) the fraction of the day spent calibrating,

including health checks. Error bars below use the 21 random seeds only; the official seed is shown separately.

Policy	Time calibrating (% of day)	Mean gate error, mean $\pm$ 95% CI (SD)	% of time below 10^{-3}	Recalibrations per day
P0	0.0015	$8.84 \times 10^{-4} \pm 1.3 \times 10^{-4}$ (2.9×10^{-4})	65.8	0
P1, 720 min	0.0018	$9.06 \times 10^{-4} \pm 1.7 \times 10^{-4}$ (3.8×10^{-4})	62.6	1
P1, 240 min	0.0030	$4.77 \times 10^{-4} \pm 6.5 \times 10^{-5}$	84.7	5
P1, 120 min	0.0048	$3.12 \times 10^{-4} \pm 3.5 \times 10^{-5}$	92.3	11
P1, 60 min	0.0086	$2.15 \times 10^{-4} \pm 2.3 \times 10^{-5}$ (5.0×10^{-5})	96.0	23
P1, 30 min	0.0153	$1.69 \times 10^{-4} \pm 1.3 \times 10^{-5}$	98.0	47
P1, 10 min	0.0441	$1.23 \times 10^{-4} \pm 4.0 \times 10^{-6}$	99.5	143
P2, 30 min, T > 16 (40 shots)	0.0072	$2.82 \times 10^{-4} \pm 3.4 \times 10^{-5}$ (7.4×10^{-5})	95.5	7.7
P2, 30 min, T > 30 (40 shots)	0.0056	$6.53 \times 10^{-4} \pm 3.9 \times 10^{-4}$ (8.6×10^{-4})	82.1	4.0

The full set of 22 configurations is in results/policy_table.md. P1 behaves as you would expect: a shorter period always lowers the error and always costs more. Pairing the seeds, P1 at 30 minutes beats P1 at 60 minutes by 4.6×10^{-5} (CI 3.0 to 6.3×10^{-5} , $p = 8.5 \times 10^{-6}$).

Where P2 fell short.

The paired difference between P2 (30 min, T > 16) and P1 (60 min) is $+6.7 \times 10^{-5}$ in gate error (CI 3.2 to 10.2×10^{-5} , $p = 7.8 \times 10^{-4}$, 21 seeds), in exchange for a slightly smaller fraction of the day (0.0072% against 0.0086%). A health check of 80 shots costs about a sixth of a warm recalibration (about 520 shots), so checking often costs about as much as it saves. When we count health-check shots as downtime, as the problem implies, P2 gives no clear gain. On the seed means a few P2 configurations are technically non-dominated (for example, a 30-minute check with T > 9.2 costs 0.0085% of the day against 0.0086% for P1 at 60 minutes, with a gate error of 2.41×10^{-4} against 2.15×10^{-4}), but those cost differences are far smaller than the seed-to-seed uncertainty on the cost ($\pm 0.0004\%$). For the headline P2 the trade is poor: it saves 0.0014 percentage points of the day at a price of 6.7×10^{-5} in gate error, whereas lengthening the P1 period from 60 to 120 minutes saves 0.0038 percentage points for 9.7×10^{-5} , about half the price per unit of time saved. Adding a fixed 10 s overhead to every calibration and health-check event removes P2 from the frontier entirely, leaving only P0 and P1. P2 occupies the middle of the frontier clearly only if health checks are free, for instance because they can run inside user

workloads (that comparison uses seed means, and neighbouring points are not statistically separated). Setting the threshold too high is also risky: with 40 shots the statistic tops out around 30 to 60 even for a 0.8 MHz jump, so $T > 30$ misses events and gives heavy-tailed results (SD 8.6×10^{-4}). This ceiling comes from our exploratory runs on the official seed, where $T > 30$ fired once and $T > 60$ never did.

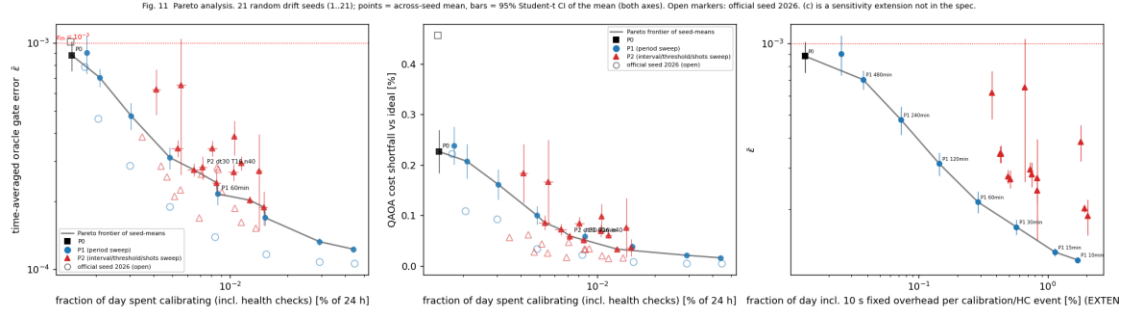


Figure 4. Pareto analysis: points are seed means, bars are 95% CIs of the mean over 21 seeds. Open markers are the official seed. Panel (c) adds a 10 s overhead per event as a sensitivity check.

Figure 5 shows one day for the official seed. P0 keeps believing the detuning is +815 kHz after the jump at 5.5 hours, and its error stays above 10^{-3} for much of the day. P1 spikes at the jump and recovers within the hour. P2 catches the jump at its next check, then recalibrates three more times, and its error creeps up to about 7×10^{-4} between recalibrations as the gain drifts.

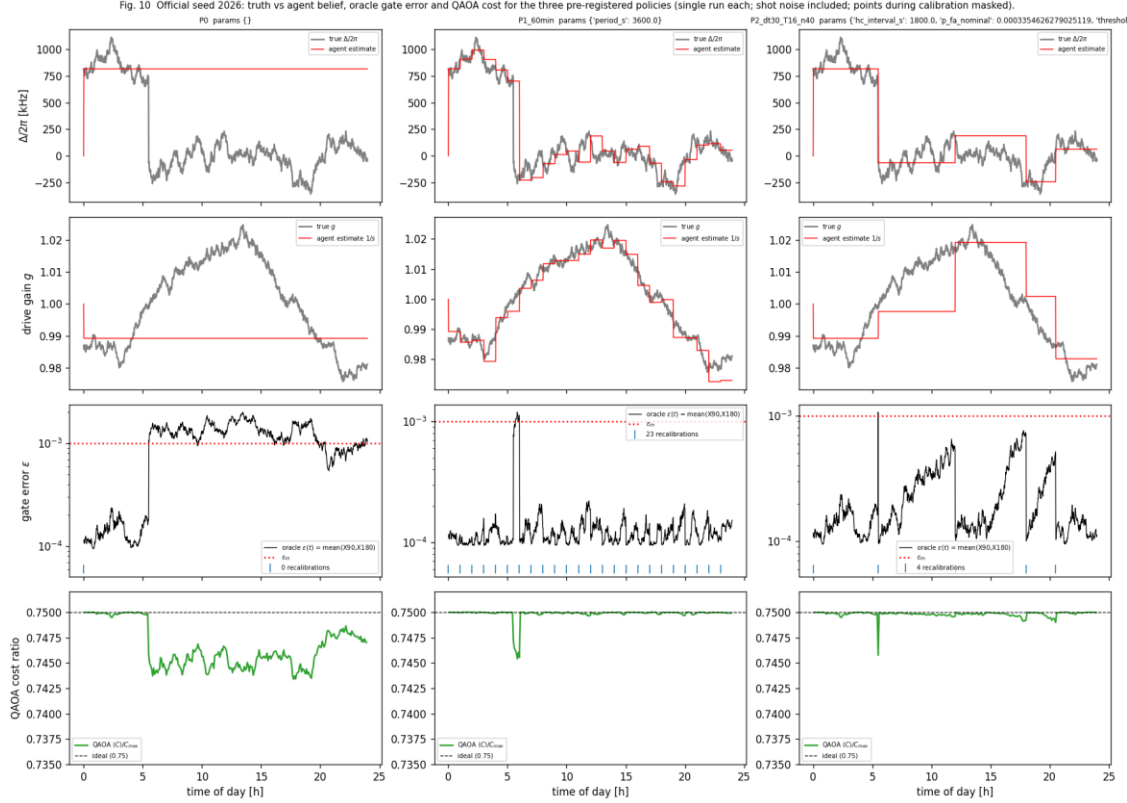


Figure 5. Official seed 2026: true values (grey) against the agent's belief (red), oracle gate error and QAOA cost. One run per policy; points during calibration are masked.

7. From gate error to QAOA cost

The QAOA is $p = 1$ MaxCut on a 4-ring. With ideal gates the best cost ratio is $\langle C \rangle / C_{\max} = 0.75$ at $(\gamma, \beta) = (\pi/4, \pi/8)$, which we checked against a direct statevector calculation. We keep those angles fixed for all hardware runs. State preparation uses $R_y(\pi/2)$ and the mixer uses $R_x(2\beta)$, each built as a pulse from the policy's current controls and the true $\Delta(t)$, $g(t)$ at that moment. We take the 2×2 block $M = \text{PUP}$ of each gate and apply it to every qubit through $\rho \rightarrow M\rho M^\dagger$ plus a maximally mixed term on that qubit carrying the lost population (the multi-qubit version of the map in the problem). Trace was preserved to about 10^{-9} in every run. The two-qubit RZZ gates are ideal.

The statement the problem asks for, from the 21 random seeds (95% CI of the mean; official seed in brackets). Here y includes health checks, and w is the fraction of the day within z of the ideal cost:

Policy	y (calibrating)	w for $z = 0.5\%$	w for $z = 1\%$
P0	0.0015%	88.0% (CI 82.6 to 93.4; official 44.3%)	98.4% (CI 96.3 to 100)
P1, 60 min	$0.0086\% \pm 0.0001$	97.2% (CI 96.0 to 98.4; official 98.3%)	100%

Policy	y (calibrating)	w for z = 0.5%	w for z = 1%
P2, 30 min, T > 16	0.0072% $\pm$ 0.0004	97.1% (CI 96.1 to 98.1; official 99.7%)	100%

In words: policy P1 at 60 minutes spends 0.0086% of the day calibrating and delivers a $p = 1$ QAOA cost within 0.5% of the ideal value for 97.2% of the day. Seed 2026 is an unlucky draw for P0, with a jump at 5.5 hours and an initial state at +0.8 MHz, so we would not quote it alone.

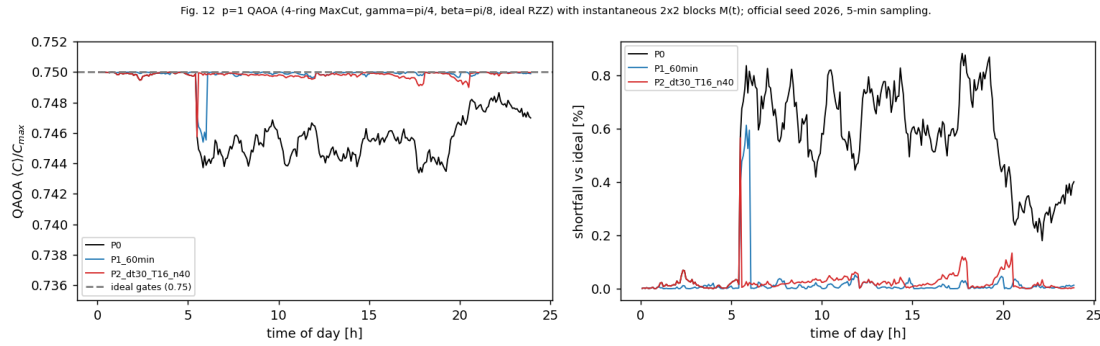


Figure 6. $p = 1$ QAOA cost over the official-seed day for the three headline policies.

Which drift matters most. We froze perfectly set controls at $t = 0$ and let one source of drift act at a time (21 seeds):

Drift source	Mean gate error	QAOA shortfall, mean (SD)
All drift	9.1×10^{-4}	0.232% (0.099)
Telegraph jump only	4.3×10^{-4}	0.176% (0.065)
OU frequency wander only	1.4×10^{-4}	0.027%
Gain (sinusoid + OU)	5.9×10^{-4}	0.061% (0.030)

The ranking depends on what you measure. Gain drift is worse for raw gate error, but the frequency jump is about three times worse for the QAOA. We suspect the detuning error acts on every state-preparation and mixer rotation, but we did not isolate that. The path from hardware to algorithm is: drift gives a residual detuning or rotation error, which sets the gate error, which accumulates over eight imperfect single-qubit gates plus the leakage map, and finally lowers $\langle C \rangle$ by a small bounded amount.

8. Testing, and what went wrong along the way

The package has 13 tests for the plant (operators, Hermiticity, the spectrum, pulse area, unitarity, matrix ordering, the two-level limit, convergence, OU, telegraph and gain statistics, reproducibility) and 18 more for the rest (oracle formulas, leakage, static π rotation, DRAG sign, units, readout, amplitude, Ramsey sign and aliasing, the posterior, adaptive behaviour, equal shots, DRAG, the information barrier, shot and time accounting, policies, health-check false alarms and power, and QAOA). All pass.

Problems we found and fixed while building it:

1. The posterior spread was dominated by far-tail tails, so we switched to a mode-based summary with an explicit tail risk.
2. The unconstrained DRAG fit sometimes locked onto a wrong fringe, so we bounded its parameters.
3. The first health check was far too sensitive and too expensive, so we redesigned it around measured gate tolerances.
4. Our first information-barrier test searched the source text and flagged a docstring, so we replaced it with a check on the parsed code, which is stricter.
5. In one demo we calibrated DRAG before frequency and got a result 0.05 ns off, which is what led us to measure and document the ordering effect.
6. We masked plot points taken during calibration, and truncated confidence intervals that exceeded 100%.

9. Limitations

- The oracle gate error is coherent only, so T_1 and T_2 affect the measurements but not the scored error. Idle detuning phase between QAOA gates is not modelled.
- The agent knows the nominal α , T_2 , readout error rates and the statistics (not the realisation) of the drift, and uses a physically bounded DRAG fit.
- Calibration costs very little in this model (at most 0.05% of the day, with health checks included), so the shot budget is rarely a binding constraint. The ordering of the policies depends on that.
- We chose the sweep ranges after looking at the official seed. Only three headline policies (P0, P1 at 60 min, P2 at 30 min with $T > 16$) were fixed before the multi-seed run. The Pareto frontier is for seed means and has no uncertainty of its own.
- The bonus tasks (a Kalman-filter tracker and a virtual-Z correction for the DRAG Stark shift) were not done.

10. References

1. We state that this work was produced with the help of AI assistants like **ChatGpt, Gemini, Claude & Perplexity**, which helped in writing the simulation and analysis code and running the experiments.
2. Pedersen, Møller and Mølmer, “Fidelity of quantum operations”, Phys. Lett. A 367, 47 (2007), the source of the fidelity formula in Eq. 3.
3. Software: Python 3.12, NumPy 2.4.4, SciPy 1.17.1, Matplotlib 3.10.8.
4. Kelly et al., “Physical qubit calibration on a directed acyclic graph”, arXiv:1803.03226 (2018).
5. Motzoi, Gambetta, Reber and Wilhelm, “Simple pulses for elimination of leakage in weakly nonlinear qubits”, Phys. Rev. Lett. 103, 110501 (2009).